POLITECHNIKA GDAŃSKA

Wydział ETI / ACIR

# OSK

Sprawozdanie z zadania 3

## Temat: Symulator stanowiska dyspozytorskiego linii produkcyjnej

**Imię i nazwisko:** Małgorzata Rogala, Cyprian Magnucki
**Numer albumu:** 198123, 197697
**Grupa:** A2A
**Przedmiot:** Organizacja systemów komputerowych
**Prowadzący:** dr inż. Krzysztof Cisowski

Gdańsk, 2026

# Spis treści

**1 Temat zadania i założenia projektowe: 2**

1.1 Temat . . . . . 2

1.2 Założenia projektowe . . . . . 2

**2 Opis działania aplikacji 3**

2.1 Logowanie . . . . . 3

2.2 Panel główny . . . . . 4

2.3 Stan urządzeń . . . . . 4

2.4 Stan produkcji . . . . . 5

2.5 Awarie i komunikaty . . . . . 5

2.6 Stan komputera . . . . . 6

**3 Opis zastosowanych rozwiązań programowych 6**

3.1 Wybór języka i biblioteki . . . . . 6

3.2 Struktura programu . . . . . 6

3.3 Przykładowe funkcje programu . . . . . 7

**4 Symulacja awarii i autodiagnostyka operatora 7**

4.1 Symulacja awarii . . . . . 7

4.2 Autodiagnostyka operatora . . . . . 8

**5 Dyskusja wyników 8**

5.1 Zalety i wady aplikacji . . . . . 8

5.2 Możliwe kierunki rozwoju . . . . . 9

# 1. Temat zadania i założenia projektowe:

## 1.1. Temat

Aplikacja symulująca stanowisko dyspozytorskie linii produkcyjnej z elementami diagnostyki procesu oraz autodiagnostyki operatora.

## 1.2. Założenia projektowe

Parametry urządeń rzeczywistej linii produkcyjnej są generowane programowo z użyciem liczb losowych, timerów oraz wybranych danych diagnostycznych komputera. Aplikacja została wykonana jako program okienkowy z interfejsem graficznym w bibliotece Tkinter.

W skład symulowanego stanowiska weszły następujące elementy:

- silnik główny linii,
- pompa chłodnicza 1, pompa chłodnicza 2,
- wentylator 1, wentylator 2, filtr powietrza,
- zasilanie główne i awaryjne,
- drzwi bezpieczeństwa, czujnik obecności operatora,
- napęd pneumatyczny.

Założono, że program będzie:

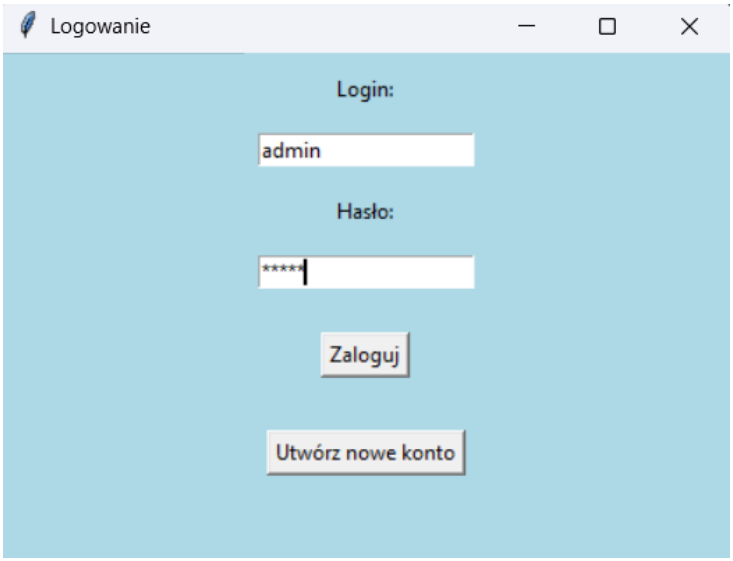
- umożliwiał logowanie operatora,
- przedstawiał aktualny stan urządzeń,
- symulował parametry pracy, takie jak temperatura, obroty, wydajność i zużycie energii,
- losowo generował awarie i stany alarmowe,
- automatycznie reagował na przekroczenia wartości dopuszczalnych,
- okresowo sprawdzał obecność operatora poprzez konieczność naciśnięcia klawisza Enter.

Wartości temperatur, obrotów i wydajności są obliczane na podstawie modelu losowego. Dane dotyczące komputera, takie jak obciążenie procesora, wykorzystanie pamięci RAM i częstotliwość CPU, pobierane są z biblioteki psutil.

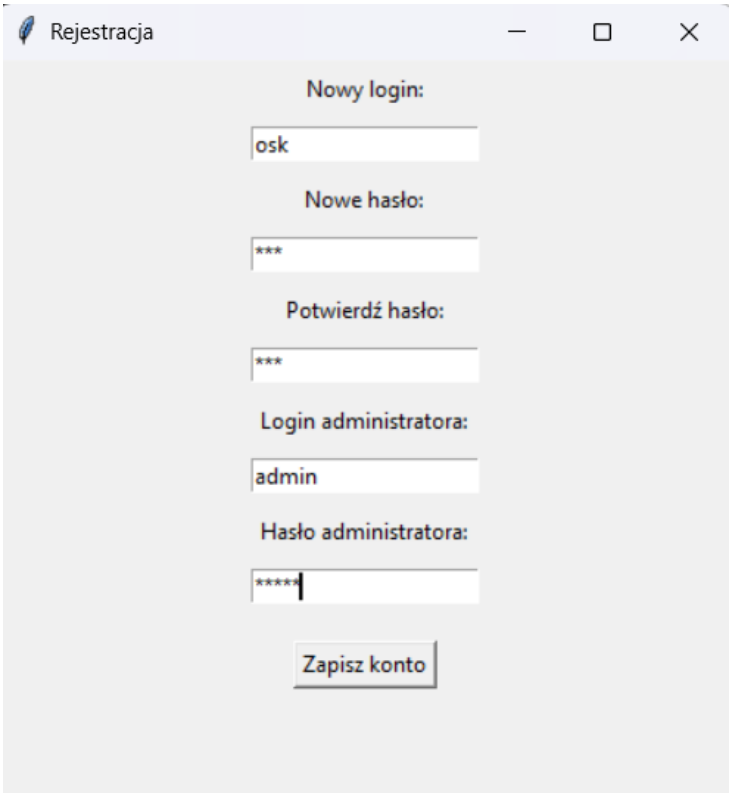
# 2. Opis działania aplikacji

## 2.1. Logowanie

Program rozpoczyna pracę od wyświetlenia okna logowania. Użytkownik może zalogować się na istniejące konto lub utworzyć nowe konto po podaniu danych administratora (login: "admin", hasło: "admin"). Po poprawnym zalogowaniu otwierany jest panel dyspozytorski.




Rysunek 1: Okno logowania




Rysunek 2: Okno rejestracji

## 2.2. Panel główny

Panel dyspozytorski zawiera menu boczne oraz część główną z widokami:

- stan urządzeń,
- stan produkcji,
- awarie i komunikaty,
- stan komputera.

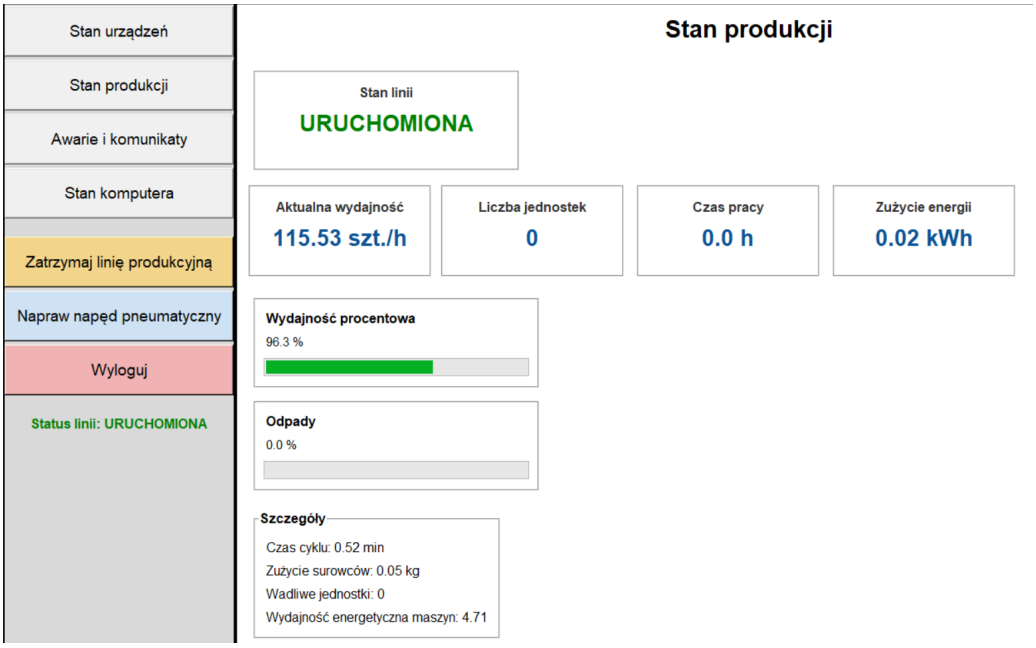
## 2.3. Stan urządzeń

W tej zakładce prezentowane są kafelki obrazujące stan podstawowych urządzeń linii. Dla silnika i pomp wyświetlana jest temperatura oraz obroty, natomiast dla wentylatorów wydajność i prędkość obrotowa. Dodatkowo pokazany jest stan elementów bezpieczeństwa i zasilania (włączone/ wyłączone).

Rysunek 3: Zakładka "Stan urządzeń"

## 2.4. Stan produkcji

W tej części aplikacja pokazuje aktualny stan pracy linii, liczbę wyprodukowanych jednostek, czas pracy, wydajność, zużycie energii, ilość wadliwych jednostek oraz poziom odpadów




Rysunek 4: Zakładka "Stan produkcji"

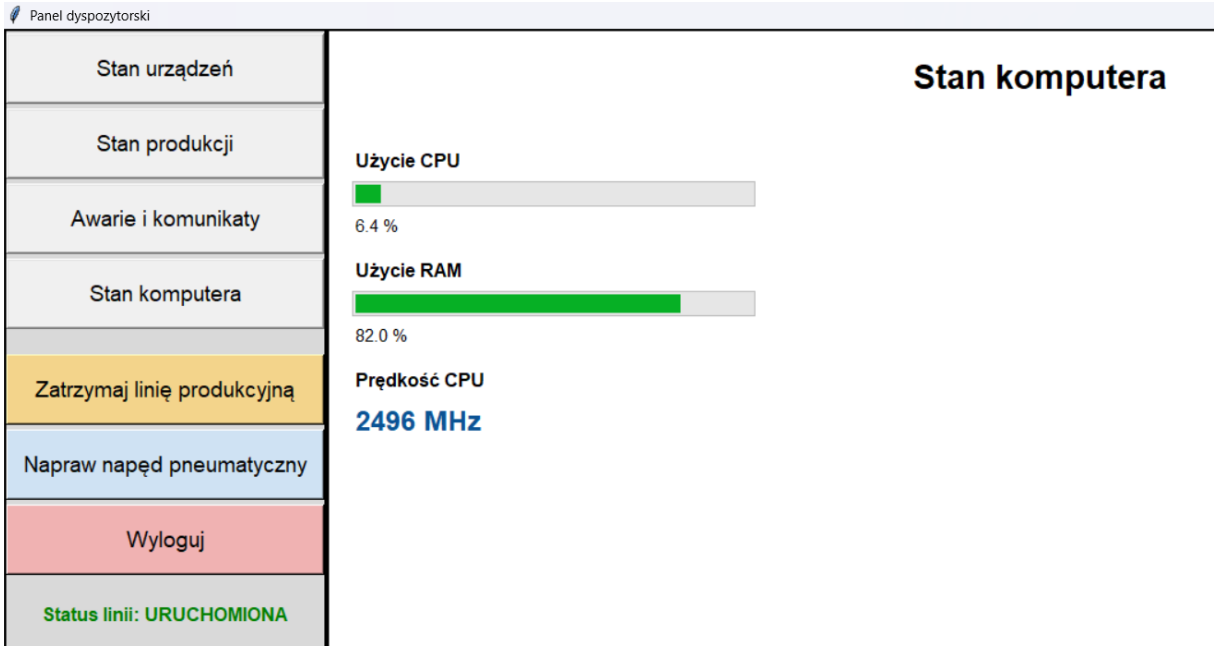
## 2.5. Awarie i komunikaty

Widok ten zawiera listę aktywnych alarmów oraz historię ostatnich zdarzeń. Operator może na bieżąco śledzić ostrzeżenia, alarmy i informacje o działaniach automatycznych systemu.

Rysunek 5: Zakładka "Awarie i komunikaty"

### 2.6. Stan komputera

Program prezentuje również dane diagnostyczne komputera, na którym działa program, takie jak użycie procesora, wykorzystanie pamięci RAM i aktualna częstotliwość CPU.




Rysunek 6: Zakładka "Stan komputera"

## 3. Opis zastosowanych rozwiązań programowych

### 3.1. Wybór języka i biblioteki

Aplikację napisano w języku Python. Interfejs graficzny wykonano przy użyciu biblioteki tkinter, natomiast do odczytu parametrów komputera wykorzystano bibliotekę psutil. Mechanizm symulacji oparto na funkcjach z modułów random, time oraz math.

### 3.2. Struktura programu

Kod programu podzielono na kilka głównych części:

- definicje użytkowników i logowania,
- moduł diagnostyki komputera,
- klasy opisujące urządzenia,
- moduł obsługi alarmów,
- panel dyspozytorski,
- moduł logowania.

Do reprezentacji urządzeń zastosowano podejście obiektowe. Utworzono klasę bazową Urzadzenie, z której dziedziczą klasy Silnik i Wentylator. Dzięki temu możliwe było uporządkowanie kodu i wydzielenie osobnych metod do aktualizacji temperatury, obrotów oraz stanu pracy.

### 3.3. Przykładowe funkcje programu

Tabela 1: Wybrane funkcje programu i ich przeznaczenie

| **Funkcja** | **Przeznaczenie** |
|---|---|
| `sprawdz_logowanie()` | sprawdza poprawność loginu i hasła użytkownika |
| `otworz_panel_dyspozytorski()` | otwiera główny panel sterowania systemem |
| `przelacz_linie()` | uruchamia lub zatrzymuje linię produkcyjną |
| `aktualizuj_urzadzenia()` | aktualizuje stan urządzeń w systemie |
| `sprawdz_i_obsluz_awarie()` | wykrywa i obsługuje awarie systemu |
| `steruj_pompami_chlodniczymi()` | steruje pracą pomp chłodniczych |
| `dodaj_zdarzenie()` | zapisuje zdarzenia w historii systemu |
| `symulacja_awarii()` | symuluje występowanie awarii i alarmów |
| `petla_glowna_aktualizacji()` | odpowiada za cykliczne odświeżanie działania programu |

## 4. Symulacja awarii i autodiagnostyka operatora

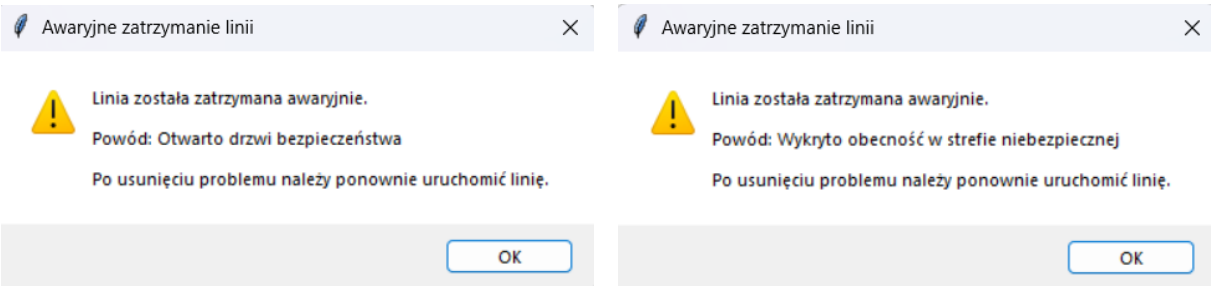
### 4.1. Symulacja awarii

W aplikacji zaimplementowano obsługę sytuacji awaryjnych oraz stanów ostrzegawczych. Część z nich wynika z przekroczenia progów parametrów, a część jest generowana losowo. Do najważniejszych zdarzeń należą:

- przekroczenie temperatury silnika,
- przekroczenie temperatury pompy chłodniczej,
- zbyt niskie obroty wentylatora,
- awaria zasilania głównego,
- otwarcie drzwi bezpieczeństwa,
- wykrycie obecności operatora w strefie niebezpiecznej,
- awaria napędu pneumatycznego,
- nadmierne obciążenie CPU komputera.

W przypadku wykrycia niektórych stanów program podejmuje działania automatyczne np.:

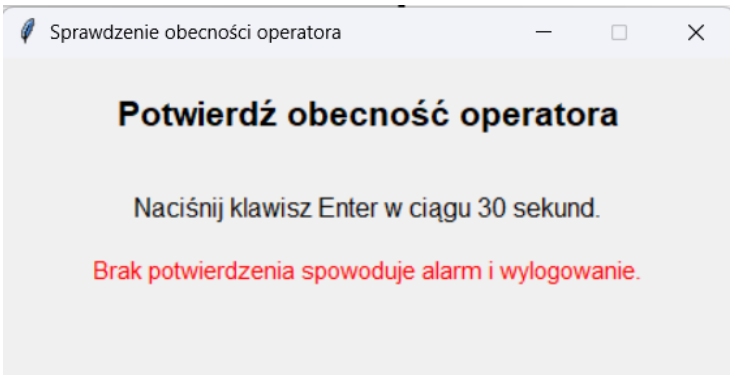
- przy wzroście temperatury silnika włączany jest drugi wentylator,
- przy wysokiej temperaturze pomp lub silnika zmniejszane jest tempo pracy linii,
- przy otwarciu drzwi bezpieczeństwa lub wykryciu obecności operatora linia zostaje zatrzymana awaryjnie,
- przy awarii zasilania głównego załączane jest zasilanie awaryjne.




Rysunek 7: Przykładowe komunikaty o awarii

### 4.2. Autodiagnostyka operatora

Co określony czas wyświetlane jest okno wymagające potwierdzenia obecności. Jeśli operator nie naciśnie klawisza Enter w ciągu 30 sekund, uruchamiany jest alarm i następuje wylogowanie z systemu.




Rysunek 8: Okno potwiwerdzenia obecności

## 5. Dyskusja wyników

### 5.1. Zalety i wady aplikacji

Do zalet programu należy czytelny interfejs, podział na logiczne widoki, obsługa alarmów, historia zdarzeń oraz połączenie diagnostyki procesu z kontrolą obecności operatora.

Wadą rozwiązania jest fakt, że parametry procesu są jedynie symulowane, a nie pochodzą z rzeczywistych urządzeń. Program ma więc charakter demonstracyjny. Dodatkowo częste odświeżanie danych może wpływać na płynność działania interfejsu. Zastosowano jednak częściową optymalizację poprzez nieblokujący odczyt użycia CPU.

### 5.2. Możliwe kierunki rozwoju

- wprowadzenie zapisu historii zdarzeń i parametrów procesu do plików lub bazy danych w celu późniejszej analizy i tworzenia raportów
- rozbudowa systemu logowania o różne poziomy uprawnień użytkowników, np. operator, serwisant i administrator
- dodanie wykresów zmian parametrów w czasie, co pozwoliłoby łatwiej obserwować trendy i wykrywać nieprawidłowości w pracy systemu
- rozbudowa programu o współpracę z rzeczywistymi urządzeniami pomiarowymi, takimi jak czujniki temperatury, ciśnienia lub sterowniki PLC, dzięki czemu aplikacja mogłaby monitorować rzeczywisty proces technologiczny